Introduction to OpenGL 1

1.1 What is OpenGL?

OpenGL is a *graphics rendering* API whose version 1.0 was released by Silicon Graphics Inc. in July 1994, and currently it is maintained by the Khronos Group (`www.khronos.org`). It provides the support for generating and displaying high-quality color images composed of geometric and image primitives. It is windowing system and operating system independent.

Although the above mentioned independence it needs some basic support from them. Different platforms have specific implementations. These implementations are known as WGL, AGL and GLX, respectively for Windows, Mac OS X and unix-based systems. There are a number of libraries that provide extra functionalities:

- GLU (OpenGL Utility Library) which is part of OpenGL provides among other things quadric shapes.
- GLUT (OpenGL Utility Toolkit), is not an official part of OpenGL, provides a portable windowing API.
- SDL (Simple Direct Media Layer), provides support for 2D graphics, OpenGL, windowing, input devices, etc.

OpenGL ES and WebGL

The success of OpenGL as a 3D graphics rendering framework attracted the interest of the mobile devices developers, as these have reached a computing power performance enough to support that. With the advent of portable computing, the embedded graphics became a real need for the handheld devices to support graphics-based applications, e.g. games, or augmented

reality. These new platforms have brought new challenges and performance concerns. This led to the definition of a new standard more adequate for the lower computing power available. OpenGL ES 1.0 appeared in 2003 as an adaptation of OpenGL 1.3 by removing some of the "heavy" functionalities from the application programming interface (API). Nevertheless the same programming principles were kept, and that led to an easy adaptation to many of the already developed applications.

Thanks to the increasing computing power of the JavaScript support in web bowsers, WebGL 1.0 was a natural consequence. In fact it was based on the OpenGL ES 2.0 standard and demonstrated to be flexible enough to support a large number of graphical applications to run directly on a web browser. WebGL 2.0 is based on the OpenGL ES 3.0 API, but it is not yet supported but some browsers.

What will I use on this course?

The short answer is WebGL!

Note, that the similarity with OpenGL allows you to explore it as well, but you should be aware that the assignments will be primarily on WebGL. My recommendation is that you directly jump to section 1.5 now, and return later to satisfy your curiosity.

As the following material about the installation and introduction to the SDKs that support the development were already written, the following sections were kept for your future reference. This includes three different ways of develop OpenGL programs: using SDL, GLUT libraries in C/C++ or Python.

1.2 OpenGL with SDL

In this case, the first step is to install the necessary libraries that support the development in C/C++ or Python. Taking Linux/Ubuntu as an example, you may use the following commands. **This is not needed for WebGL.**

```
Linux $> sudo apt install python-pip freeglut3-dev libsdl2-dev
Linux $> pip install PyOpenGL PyOpenGL_accelerate
```

The following code is a very basic skeleton of an OpenGL program using SDL to communicate with the windowing system.

— C++ —

```
#include <stdio.h>
#include <stdint.h>
#include <assert.h>
#include <SDL2/SDL.h>
#include <SDL2/SDL_opengl.h>

#ifdef __APPLE__
#include <OpenGL/gl.h>
#include <OpenGL/glu.h>
#else
#include <GL/gl.h>
#include <GL/glu.h>
#endif

typedef int32_t i32;
typedef uint32_t u32;
typedef int32_t b32;

#define WinWidth 640
#define WinHeight 480
SDL_Window *Window;

void InitGL(int Width, int Height) {
    glClearColor( 0.0, 0.0, 0.0, 1.0 );
    glClearDepth( 1.0 );
    glDepthFunc(GL_LESS);
    glEnable(GL_DEPTH_TEST);
    glShadeModel(GL_SMOOTH);

    // You must insert here data structures initialization
    // namely those that support the models' vertices, colors, etc.
    // And compute the necessary matrices (projection, view) that are
    // not going to change (frequently)
}

void ReSizeGLScene(int Width, int Height){
    glViewport(0, 0, Width, Height);

    // You will need to recompute the projection matrix,
    // in particular for accounting for changes on the
    // aspect ratio.
}

void DrawGLScene( void ) {
  glClear( GL_COLOR_BUFFER_BIT  | GL_DEPTH_BUFFER_BIT);

  // YOUR SCENE DRAWING CODE GOES HERE

  SDL_GL_SwapWindow(Window);
}

int main (int ArgCount, char **Args) {

    u32 WindowFlags = SDL_WINDOW_RESIZABLE | SDL_WINDOW_OPENGL;
    Window = SDL_CreateWindow("CG&RA SDL", 0, 0,
                              WinWidth, WinHeight, WindowFlags);
    SDL_GLContext Context = SDL_GL_CreateContext(Window);

    b32 Running = 1;
    b32 FullScreen = 0;
```

```
61
62      InitGL(WinWidth,WinHeight);
63
64      while (Running)  { // Main loop
65          SDL_Event ev;
66          while (SDL_PollEvent(&ev)) {
67              switch(ev.type) {
68              case SDL_QUIT:
69                  Running=false;
70                  break;
71              case SDL_WINDOWEVENT:
72                  switch (ev.window.event) {
73                  case SDL_WINDOWEVENT_RESIZED:
74                      SDL_Log("Window %d resized to %dx%d",
75                              ev.window.windowID, ev.window.data1,
76                              ev.window.data2);
77                      ReSizeGLScene(ev.window.data1, ev.window.data2);
78                      break;
79                  } break;
80
81              case SDL_KEYDOWN:
82                  switch (ev.key.keysym.sym){
83                  case  SDLK_ESCAPE:
84                  case SDLK_q:
85                      exit(0);
86                  case SDLK_f:
87                      FullScreen = !FullScreen;
88                      if (FullScreen) {
89                          SDL_SetWindowFullscreen(Window, WindowFlags |
90                                                  SDL_WINDOW_FULLSCREEN_DESKTOP);
91                      }
92                      else {
93                          SDL_SetWindowFullscreen(Window, WindowFlags);
94                      }
95                      break;
96                  }
97              }
98          }
99
100         DrawGLScene();
101     } // Main loop
102     return 0;
103 }
```

The event processing switch clauses can be extended, as there are many different events that are supported by SDL. Hereafter follows an example piece of code that could be used to extend the events supported by the application.

— C++ —

```
1  ...
2      case SDL_WINDOWEVENT:
3          switch (ev.window.event) {
4          case SDL_WINDOWEVENT_SHOWN:
5            SDL_Log("Window %d shown", ev.window.windowID);
6            break;
7          case SDL_WINDOWEVENT_HIDDEN:
8            SDL_Log("Window %d hidden", ev.window.windowID);
9            break;
```

```
10          case SDL_WINDOWEVENT_EXPOSED:
11            SDL_Log("Window %d exposed", ev.window.windowID);
12            break;
13          case SDL_WINDOWEVENT_MOVED:
14            SDL_Log("Window %d moved to %d,%d",
15                    ev.window.windowID, ev.window.data1,
16                    ev.window.data2);
17            break;
18          case SDL_WINDOWEVENT_RESIZED:
19            SDL_Log("Window %d resized to %dx%d",
20                    ev.window.windowID, ev.window.data1,
21                    ev.window.data2);
22            ReSizeGLScene(ev.window.data1, ev.window.data2);
23            break;
24          case SDL_WINDOWEVENT_MINIMIZED:
25            SDL_Log("Window %d minimized", ev.window.windowID);
26            break;
27          case SDL_WINDOWEVENT_MAXIMIZED:
28            SDL_Log("Window %d maximized", ev.window.windowID);
29            break;
30          case SDL_WINDOWEVENT_RESTORED:
31            SDL_Log("Window %d restored", ev.window.windowID);
32            break;
33          case SDL_WINDOWEVENT_ENTER:
34            SDL_Log("Mouse entered window %d",
35                    ev.window.windowID);
36            break;
37          case SDL_WINDOWEVENT_LEAVE:
38            SDL_Log("Mouse left window %d", ev.window.windowID);
39            break;
40          case SDL_WINDOWEVENT_FOCUS_GAINED:
41            SDL_Log("Window %d gained keyboard focus",
42                    ev.window.windowID);
43            break;
44          case SDL_WINDOWEVENT_FOCUS_LOST:
45            SDL_Log("Window %d lost keyboard focus",
46                    ev.window.windowID);
47            break;
48          case SDL_WINDOWEVENT_CLOSE:
49            SDL_Log("Window %d closed", ev.window.windowID);
50            break;
51          default:
52            SDL_Log("Window %d got unknown event %d",
53                    ev.window.windowID, ev.window.event);
54            break;
55          } break;
56
57        case SDL_MOUSEBUTTONDOWN:
58          switch (ev.button.button){
59          case SDL_BUTTON_LEFT:
60            SDL_Log("Left mouse button down");
61            break;
62          case SDL_BUTTON_RIGHT:
63            break;
64          }
65          break;
66
67        case SDL_MOUSEBUTTONUP:
68          switch (ev.button.button){
69          case SDL_BUTTON_LEFT:
70            SDL_Log("Left mouse button up");
71            break;
```

```
72            case SDL_BUTTON_RIGHT:
73              break;
74            }
75            break;
76
77          case SDL_MOUSEMOTION:
78            break;
79  ...
```

The above code can be compiled on Ubuntu with

```
Linux $>   g++ opengl_sdl.cpp -lSDL2 -lGLU -lGL
```

On Windows with msvc

```
C:\me >  cl sdl2_opengl.cpp /I C:\sdl2\include /link C:\sdl\SDL2.lib
C:\me >  C:\sdl\SDL2main.lib /SUBSYSTEM:CONSOLE
C:\me >  /NODEFAULTLIB:libcmtd.lib opengl32.lib
```

and on Mac OS X with gcc and SDL2 installed via macports

```
Mac $>   gcc -I/opt/local/include sdl2_opengl.cpp -L/opt/local/lib
      -lSDL2    -framework OpenGL
```

1.3 GLUT

The same can be implemented using GLUT as follows

— C++ —

```
1  #ifdef __APPLE__
2  #include <OpenGL/gl.h>
3  #include <OpenGL/glu.h>
4  #else
5  #include <GL/gl.h>
6  #include <GL/glu.h>
7  #endif
8
9  #include <GL/glut.h>
10 #include <stdio.h>
11
12 void InitGL(int Width, int Height) {
13   glClearColor( 0.0, 0.0, 0.0, 1.0 );
14   glClearDepth( 1.0 );
15   glDepthFunc(GL_LESS);
16   glEnable(GL_DEPTH_TEST);
17   glShadeModel(GL_SMOOTH);
18     // You must insert here data structures initialization
```

```
19      // namely those that support the models' vertices, colors, etc.
20      // And compute the necessary matrices (projection, view) that are
21      // not going to change (frequently)
22  }
23
24  void ReSizeGLScene(int Width, int Height){
25    glViewport(0, 0, Width, Height);
26      // You will need to recompute the projection matrix,
27      // in particular for accounting for changes on the
28      // aspect ratio.
29  }
30
31  void DrawGLScene( void ) {
32    glClear( GL_COLOR_BUFFER_BIT  | GL_DEPTH_BUFFER_BIT);
33
34     // YOUR SCENE DRAWING CODE GOES HERE
35
36    glutSwapBuffers();
37  }
38  void keyPressed( unsigned char key, int x, int y ) {
39    switch( key ) {
40    case 27:
41    case 'q' :
42    case 'Q' :
43      exit( EXIT_SUCCESS );
44      break;
45    }
46  }
47
48  void idle( void ) {
49    glutPostRedisplay();
50  }
51
52  int main( int argc, char** argv ) {
53    glutInit(&argc, argv);
54
55    glutInitDisplayMode( GLUT_RGBA|GLUT_DOUBLE|GLUT_DEPTH );
56    glutInitWindowSize(640,480);
57    glutInitWindowPosition(0,0);
58    glutCreateWindow("CG&RA GLUT");
59    glutDisplayFunc( DrawGLScene );  // register callback routines
60    glutReshapeFunc( ReSizeGLScene );
61    glutKeyboardFunc( keyPressed );
62    glutIdleFunc( idle );
63    InitGL(640,480); // initiate OpenGL states, program variables
64    glutMainLoop(); // enter the event-driven loop
65  }
```

The above code can be compiled on Ubuntu with

```
Linux $>   g++ opengl_glut.cpp -lGLUT -lGLU -lGL
```

For better dealing with the build-edit process it is advised to write a Makefile as follows, and then use the make command on a terminal.

— C++ —

```
1  all: opengl_sdl opengl_glut
```

```
2
3  opengl_sdl: opengl_sdl.cpp
4          g++ opengl_sdl.cpp -lSDL2 -lGLU -lGL -o opengl_sdl
5
6  opengl_glut: opengl_glut.cpp
7          g++ -o opengl_glut opengl_glut.cpp -lGLU -lglut -lGL
8
9  clean:
10         rm -r opengl_sdl opengl_glut *~
```

1.4 Python with PyOpenGL

PyOpenGL is available as a module installable via pip command and requires GLUT or other supported windowing library to be installed.

For other operating systems check http://pyopengl.sourceforge.net/ website.

The following code is similar to the previous versions in C, but this time in python using GLUT.

— C++ —

```
1  from OpenGL.GL import *
2  from OpenGL.GLUT import *
3  from OpenGL.GLU import *
4  import sys
5
6  ESCAPE = '\033'
7  window = 0
8
9  def InitGL(Width, Height):
10     glClearColor(0.0, 0.0, 0.0, 0.0)
11     glClearDepth(1.0)
12     glDepthFunc(GL_LESS)
13     glEnable(GL_DEPTH_TEST)
14     glShadeModel(GL_SMOOTH)
15
16     # You must insert here data structures initialization
17     # namely those that support the models' vertices, colors, etc.
18     # And compute the necessary matrices (projection, view) that are
19     # not going to change (frequently)
20
21
22 def ReSizeGLScene(Width, Height):
23     glViewport(0, 0, Width, Height)
24
25     # You will need to recompute the projection matrix,
26     # in particular for accounting for changes on the
27     # aspect ratio.
28
29
30 def DrawGLScene():
31     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
32
33     # You must insert here data structures initialization
34     # namely those that support the models' vertices, colors, etc.
35     # And compute the necessary matrices (projection, view) that are
```

```
36      # not going to change (frequently)
37
38      glutSwapBuffers()
39
40  def keyPressed(*args):
41      global window
42      if args[0] == ESCAPE:
43          sys.exit()
44
45  def idle():
46       glutPostRedisplay()
47
48  def main():
49      global window
50      glutInit(sys.argv)
51
52      glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE | GLUT_DEPTH)
53      glutInitWindowSize(640, 480)
54      glutInitWindowPosition(0, 0)
55      window = glutCreateWindow("CG&RA Python")
56
57      glutDisplayFunc(DrawGLScene)
58
59      glutIdleFunc(DrawGLScene)
60      glutReshapeFunc(ReSizeGLScene)
61      glutKeyboardFunc(keyPressed)
62      InitGL(640, 480)
63      glutMainLoop()
64
65  # Print message to console, and kick off the main to get it rolling.
66  print "Hit ESC key to quit."
67  main()
```

It can be executed by typing on the terminal

```
Linux $> python opengl_py.py
```

1.5 WebGL

WebGL is an adaptation of OpenGL to be used on web pages. But as we are going to use Jupyter Notebooks to support the development you will need to pay attention to the following:

1. Contrary to normal Jupyter Notebook use, the results will not be seen directly bellow the cells, but instead using **Voilà**.
2. All the cells need to include `%%html` in the first line
3. You may use HTML tags along with JavaScript

The WebGL canvas

GL needs a dedicated space to draw the scene, and above there were examples on how to open specialised windows for OpenGL in a C++ or Python application. Here we will do the same inside a browser, but we will call it a canvas.

— **HTML** —

```
<canvas id="myCanvas" width="400" height="400" style="border:2px solid #000000;">
      Error: Your browser does not support the HTML canvas tag.
</canvas>
<script id="myapp">
 // Your JavaScript code goes here
</script>
```

The above HTML code instructs the browser to create a canvas with 400 by 400 pixels and give it an identification id="myCanvas". It is through this "id" that we will refer to it latter in our code. The script element will contain your JavaScript code that will support your GL scene rendering.

To simplify your learning task and make a parallel with the Python and C++ versions, you are going to use a library of classes already available. One is responsible for "shader" management, and the other supports the "application" itself. As you can see on the example notebook, there is a cell responsible for preloading those classes, with:

— **HTML** —

```
<script src="https://is3l.isr.uc.pt/~pm/CGRA/JS/deecshader.js" ></script>
<script src="https://is3l.isr.uc.pt/~pm/CGRA/JS/deecapp.js" ></script>
```

There are also two cells, one with the Vertex Shader code, and the other with the "Fragment Shader" code, but we will discuss them later.

— JavaScript —

```
class myapp extends DEECapp{
    counter = 0;
    initialize(){
        super.initialize();
        this.gl.clearColor(1.0,1.0,1.0,1.0);

        // You must insert here data structures initialization
        // namely those that support the models' vertices, colors, etc.
        // And compute the necessary matrices (projection, view) that are
        // not going to change (frequently)
    }

    render(){
        console.log('draw' +  counter);
        counter++;
        this.gl.clear(this.gl.COLOR_BUFFER_BIT | this.gl.DEPTH_BUFFER_BIT);

          // YOUR SCENE DRAWING CODE GOES HERE
    }
}

var app = new myapp('myCanvas');
app.run();
```

Now you just have to pay attention to this class in the given example. There are two main methods: initialize(), for setting the necessary data up, and render() that will be called whenevet we need to update the view. Note that the underlying class guarantees that this render method is being called about 50 times per second, as you will understand later. Another important issue is that every "GL" related function or constant is to be used in this case as `this.gl.functionname(..)` or `this.gl.CONSTANTNAME`.

1.6 Development of exercises and assignments

A computer cluster was configured to support the lab exercises and assignments. For accessing it you must configure and use OpenVPN as indicated on the following address `https://helpdesk.deec.uc.pt/help/pt-pt/19-vpn`.

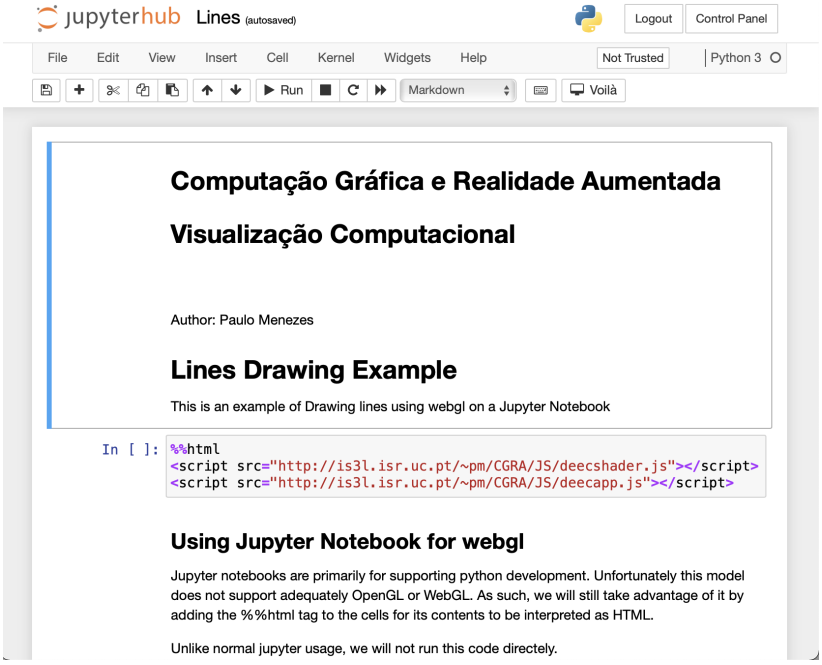
After configuring this access you should be able to access the JupyterHub at `https://seeds.deec.uc.pt/CGRA/` where you should authenticate yourself using the Inforestudante[1] credentials (mail and password). Upon successful authentication you should see the listing of your workspace as shown in the image bellow.

[1]For students from degrees other than MEEC, you should receive a mail soon from DEEC network support services. Pay attention to your official email inbox.



Currently you will have an examples folder, where you will always find the latest examples. As its contents will be updated each time you log in, you should copy the example that you are going to work on to another folder, otherwise your changes will be lost.

After copying the first example, you may now open it read it carefully.



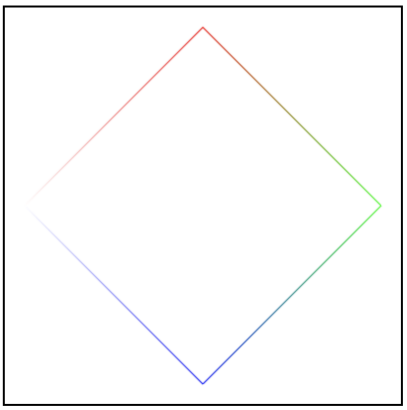
Important notice: Do not try to click the run button as you will get unpredictable results. Instead of that, click on the **Voilà** button, and a new tab or window should pop up. Eventually you may need to authorise your browser to display it.

Now, scrolling the new page to its bottom, you should see the following.

```
        this.gl.drawArrays(this.gl.LINE_LOOP,0,this.numvertices);
        this.shaderprog.stopUsing();
    }
}

var app = new myapp('myCanvas');

app.run();
```



This is the output of the first GL program that you are going to explore.

1.7 Exercises

Exercise 1.1
Explore the Lines example notebook that you will find on the `examples` folder on your workspace. Read it carefully and in particular try to understand where are the coordinates of the triangle vertices, and respective colours.

1. Change the colours and see what happens. For that you should click the "save" button and if you have already opened the "voilà" tab, just reload it.

2. Change the coordinates of one vertex and if you succeed change other ones. Note that the coordinates must be in the range $[-1..1]$.

Exercise 1.2
Discover the javascript console on your web browser. You may notice that there is a message being output there. How does it relate with the changing background?

Exercise 1.3

Add one more point to the figure. If you don't get a picture, check on the web/javascript console what is happening. You may discover and explore the debugging tools there.

Exercise 1.4
On the call to the drawArrays function, change the graphics primitive to render from gl.LINE_LOOP to each of the following and observe the results, if any: gl.POINTS, gl.LINE_STRIP, gl.LINES, gl.TRIANGLE_STRIP, gl.TRIANGLE_FAN, gl.TRIANGLES.

Note that in one of the main differences between WebGL and OpenGL is that the functions name in the C/C++ API are of the form glXxxxXxxx (e.g. glDrawArrays) and constants are defined as GL_XXX_XXX, where in WebGL we will define a context variable (gl or this.gl) and then the functions will be used as gl.xxxXxxx or this.gl.xxxXxxx (in our case).